Applying KSE-test and K-means clustering towards Scalable Unsupervised Intrusion Detection

Thanachai Jirachan
Dept. of Computer Engineering
Chulalongkorn University
Bangkok, Thailand
Email: thanachai.ji@student.chula.ac.th

Krerk Piromsopa
Dept. of Computer Engineering
Chulalongkorn University
Bangkok, Thailand
Email: krerk.p@chula.ac.th

***Abstract*— We proposed a scalable outlier detection method to identify outliers in large datasets with a goal to create unsupervised intrusion detection. In our work, the strength of Kolmogorov-Smirnov test and K-means clustering algorithm, both with linear time complexity, are combined to create fast outlier detection. While still maintaining high detection rate and low false alarm rate, our method can easily be paralleled for processing a large data set. The result is then applied with a predefined threshold in order to create efficient intrusion detection. We validated our method against the KDD'99 dataset. With appropriate value of threshold and value of K in KSE test, the results showed the detection rate up to 80% with false alarms less than 10%. While scaling linearly, the accuracy of our method is also improved from those of pure KSE-test-based methods.**

Keywords- Outlier Detection, Log analysis, Anomaly detection, Intrusion detection, Clustering

I. INTRODUCTION

In recent years, malicious intrusions have wrecked havoc on numerous computer systems. Intrusions are malicious actions directed at systems that may cause problems to configurations, information, data or services of the systems. The eventual result is that the systems cannot perform in normal states or might even caused these systems to be invaded by attackers. To detect intrusions, we need efficient rules to apply to an Intrusion Detection System (IDS) [1] for monitoring and notifying any suspicious activities that occur in the systems.

There are many approaches for identifying network intrusions from a large amount of usage data in log files. Without the requirement of prior knowledge, anomaly detection is one of the most effective techniques for identifying unknown intrusive behaviors. Since misuse detection techniques need based knowledge about the characteristics of data from training dataset to classifying data, anomaly detection techniques can step over that limitation by using a general assumption that the suspicious instances are far more deviated from normal instances.

There are many techniques for identifying anomalies [2,3]. Clustering is one of the unsupervised learning techniques that can classify large datasets by dividing data into small chunks, each of which contains similar instances in a cluster. These techniques are generally used to analyze large data with high dimensions. In addition, clustering techniques are unsupervised learning that do not require previous knowledge to classify data. To extract the anomalous instances from the clustering result, small size clusters and noises are marked as anomalies.

Outlier detection [4] is an approach that is specifically designed to find the outliers that deviated from other data in datasets. There are many unsupervised approaches such as statistical-based, distance-based, density-based and high-dimensional approach.

Statistical-based methods are methods that based on statistical methods and statistical tests. The output is anomaly scores of each instance. An example is the Histogram-based Outlier Score (HBOS) algorithm [5]. The advantages of statistical approach are speed and lower resource usages when compared to other approaches. However, simple statistical methods may not yield a good detection rate.

Distance-based outlier detection approaches detect outlier by using distances from each instance to its neighbors. The base assumption are: normal instances are close to their neighbors while outlier instances are much further from their neighbors. Examples are approaches in [6,7,8] and K nearest neighbor (KNN) based approaches [9,10,11]. The naïve KNN-distance based approaches, with the complexity of $O(n^2)$, are struggle to cope with large datasets.

Density-based outlier detection [12,13,14] assumes that normal instances are located in high-density area, and outliers are in the sparse area. Thus, they have to compare the density of each point. Since most of density-based approaches have high complexity (Exponential runtime), they were difficult to deal with large datasets and high-dimensional data.

From the explanation, anomaly detection approaches are useful in detecting unknown intrusions. In practice, they still suffer from high false-detection rates and low detection rates. In addition, they are not efficient enough for large data size. Although some approaches have linear complexity, they often have yield low-quality results. Basically, we have to trade-off between detection accuracy and speed.

978-1-4799-1966-6/15/$31.00 ©2015 IEEE

Authorized licensed use limited to: INSTITUTE OF ENGINEERING & MANAGEMENT TRUST. Downloaded on April 24,2026 at 14:46:40 UTC from IEEE Xplore. Restrictions apply.

The paper is organized as follows. In section 2, we review the related works and introduce KSE-test algorithm. Our proposed methodology is explained in Section 3. The experiment results are given in Section 4. Finally, we conclude our proposed algorithm and future works in Section 5.

II. Related Works

The main role of Intrusion detection system (IDS) is to analyze network activities and to discover vulnerabilities in the network. IDS data sources are logging data in the system, event the logs on each host (host-based ids) and the logs of traffics in the network (network-based).

Unsupervised anomaly detection method can be applied to analyze network activities in traffic log to help IDS identifies normal and intrusions. Many experiments try to adapt clustering algorithms for anomaly detection tasks such as [15]. Most clustering algorithms have to deal with large time complexity and have to trade-off quality of the result and speed. K-means algorithm is a light-weight clustering approach that has linear time complexity. For this reason, many experiments try to modify K-means clustering so that it can be used to detect anomalies.

A. K-means clustering algorithm

K-means clustering algorithm is well-known unsupervised clustering algorithm. It can quickly categorize data into clusters by assigning data into nearest cluster. First, it chooses k initial centroid of cluster (number of clusters). Each instance is then assigned to the nearest cluster with the closest centroid determined by shortest Euclidean distances between observed instances to the centroids. Once finished, the centroid of each cluster is recalculated using the mean value of all points in the cluster. The instance is re-assigned into cluster again. This process is iterated until the centroids stop moving or reaching the maximum number of iterations. Finally, it returns the clusters of similar instances within each cluster. K-means clustering has an ability to handle high dimensional data.

Ahmed and A. Naser [4] proposed outlier detection and clustering improvement (ODC) algorithm that modified K-means to perform outlier detection by scoring outlier degree of each instance during iteration. The outlier degree is calculated by comparing the distance between instances and the cluster's centroid to the predefined threshold distance value. However, a threshold value does not conform to a varied cluster size and varied cluster radius.

For the next algorithm, Anomaly Intrusion Detection Method Based on K-Means Clustering Algorithm with Particle Swarm Optimization (PSO-KM) algorithm [16], the researchers improve K-means algorithm by applying Particle Swamp Optimization (PSO) algorithm to calculate position of the centroid instead of using the mean of cluster called PSO-KM algorithm to detect intrusions. This helps avoiding the impact of selecting initial cluster and overcome the limitation of the traditional k-means algorithm that easily falling into local minima. Their experimental results show that PSO-KM algorithm can reduce false alarm and increase detection rate because it generates highly similar clusters when it is converged. However, both experiments do not perform well for outlier detections. Since it is not primarily designed for finding outliers, the clustering algorithm is suitable for finding groups of data.

In our experiment, we apply K-means clustering to assist identifying normal instances using cluster size.

B. Statistical-based outlier detection

Statistical-based outlier detection is another choice that can efficiently find anomalies in a linear time. An example is the outlier detection using histogram-based approach (HBOS) [5]. The approach identifies types of instances whether they are normal or intrusions by scoring an outlier score. The higher score indicates a high probability that an instance can be an intrusion. It separates the intrusions from the normal instances by comparing outlier scores to a predefined threshold value. While this proposed method is fast and has low computational complexity, it still has low detection rates and high false alarm rates.

C. Two sample Kolmogorov-Smirmov test

Two-sample Kolmogorov-Smirnov test (KS-test) is a statistical test typically used to find whether two samples have the same distribution. To accept an initial hypothesis H0: F(x)=G(x), that the two samples have the same distribution, so it calculates maximum differences between two distribution functions (F(x) and G(x)) and return the score (KS-score) between 0 and 1. The score must not exceed the critical value. If it is the case, then we accept H0 with significant level α.

D. Kolmogorov-Smirnov and Efron Outlier Detection

Kolmogorov-Smirnov and Efron Outlier Detection algorithm is a KSE-test based upon resampling (Efron) [17] by applying the KS-score. The score is calculated for each instance x in dataset D using Euclidean distance from x to points in sample set S

Given a data set S that has n instances with m attributes, we assign the KSE-score an outlier score to each instance in the dataset S. A high outlier score indicates that a higher change of that point being an anomaly.

The outlier score is computed for each point in the set S using the average KS statistic, also called KSE statistic. The calculation can be explained as follows. First, the method creates a sample set to estimate empirical cumulative distribution function (ECDF), $Fp_j(x)$. Instead of calculating the average KS-score of all points in dataset S by resampling representation set of whole dataset, a defined fixed sample size has x instances. To reduce time complexity of the process, sample1 (S_1) and sample2 (S_2) are resampled with x instances reduce from n instances of dataset. Second, an estimated ECDF $Fp_j(x)$ that describes the distribution of Euclidean distances from any point j in the dataset S to any other point in the S_1 set is used to compute the KS-score between $Fp_j(x)$, distribution of Euclidean distance from point j to all point in S_1 by comparing to $Fp_i(x)$, distribution of Euclidean distances from point i in S_2 to all point in S_1. Next, the KS statistic between the point j and any other points in S_1 is computed using the equation 1.

$$KS(p_j, p_i) = MAX\left|Fp_j(x) - Fp_i(x)\right| \quad (1)$$

Authorized licensed use limited to: INSTITUTE OF ENGINEERING & MANAGEMENT TRUST. Downloaded on April 24,2026 at 14:46:40 UTC from IEEE Xplore. Restrictions apply.

Finally, by taking the average of these KS test statistics (between point j and all points in S_2), the KSE test statistic is computed and used as the outlier score using equation 2.

$$KSE(p_j) = \frac{1}{n-1}\sum_{\substack{i=1 \\ i \neq j}}^{n} KS(p_j - p_i) \qquad (2)$$

Since the KS statistic is bounded between 0 and 1, the value of KSE statistic is also bounded between 0 and 1 as well.

In conclusion, we see that clustering algorithms is not suitable for outlier detection. Many experiments have been applied to make a clustering algorithm for intrusion detections. However, the disadvantage of most unsupervised approaches is that they still suffer from high false alarm rates and low detection rates. So we propose an outlier detection approach to increase detection rates and reduce false alarm rates in network intrusion detection system.

III. Proposed Methodology

Our methodology can be separated into 4 main steps. After we have preprocessed dataset, we apply KSE-test Outlier Detection to create intrusion detection. In the second step, we applied K-means Clustering algorithm on a large data log to cluster instances into clusters. Then, we enhance an accuracy of the detection result by removing normal instances using normal profile to reduce false positive in the third step. Finally, we obtain the final label by using the majority vote. The steps of our approach are shown in figure1.

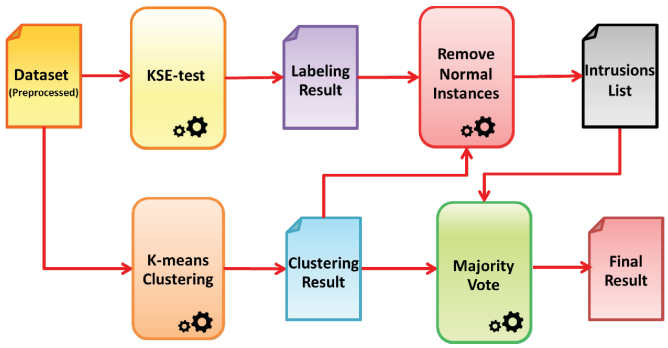

Figure 1: Data flow and result of each step

A. *Calculating KSE-score and Labeling Instances*

In this step, the preprocessed is used to calculate the anomaly score for each instance. The KSE-test is the outlier detection method, based-on Kolmogorov-Smirnov two sample test, used to compute an outlier score for each instance in datasets. We use KSE-score, average KS-score (a maximum different CDF, cumulative distribution function, from two sample Kolmogorov-Smirnov test) as an outlier score of instances to identify the probability of being intrusion. To do, we define threshold from average KS-score (result of K-S test are between 0 and 1). A fix threshold can be used to separate attacks from normal. Then we get a set of labeled data as an output. We re-implement the KSE-test algorithm to optimize for speed by reducing redundant sampling and computing of distances. Our implementation can be described in figure2.

```
function KSE_Test (dataset[],sample_size){

    sample1[] = create1DArray(sample_size)
    sample2[] = create1DArray(sample_size)
    distance_matrix[][] = create2DArray(sample_size,sample_size)
    kse_score[] = create1DArray(dataset.length)

    sample1 = randomData(dataset[],sample_size) //random sample1
    sample2 = randomData(dataset[],sample_size) //random sample2

    //Calculate distance from all points in sample1 to all points in sample2
    for(x=0 -> sample1.length){
        for(y=0 ->sample2.length){
            distance_matrix[x][y] = EuclideanDistance(instance_s1[x],instance_s2[y])
        }
    }

    //Calculate average KS-score for each instance
    for(x=0 -> dataset.length){
        observed_instance = dataset[x]
        distance_array[] = create1DArray(sample_size)
        for(y=0 ->sample2.length){
            distance_array[y] = EuclideanDistance(observed_instance,instance_s2[y])
        }
        sum = 0
        for(z=0 ->sample1.length){
            ks_score = KStest(distance_array,distance_matrix[z])
            sum += ks_score
        }
        average = sum/sample1.length
        kse_score[x] = average
    }
    return kse_score
}
```

Figure 2: KSEtest function

The KSE-test algorithm is re-implemented in JAVA. Our minor modification form original algorithm yields better speed. With the modification, we only resample data once to reduce the number of excessive distance computations. We also optimize the KS-test using a faster approach.

B. *K-means Clustering and Creating Normal Profile*

Although clustering algorithms are not suited for directly detecting outliers, it can well be used in identifying group of normal instances. Since the clustering algorithm was optimized to find the group of the similar instances rather than the outlier, we can use grouping ability to classify normal instances. To generate a normal profile, we classify data into clusters by applying K-means clustering to a dataset, so the instances that are located in the same cluster have high similarity and are different from other clusters. From the underlying assumption that the number of normal points is much large than intrusions, we label instances in large clusters as normal. Sorted by size, we create normal profile by label instances in the largest cluster as normal and repeat with the smaller clusters until the number of instances cover 80 percent of dataset as suggested by our prior experiment.

With the basic assumption of anomaly detection, the number normal instances must be far greater than the number of intrusion. We assign a cluster with large number of members as a normal cluster. So, we get a normal profile for selecting out the normal instances that were contaminated from outlier labeled instance from previous step. We will discuss the effect of the K value and explain the criteria that we used to choose the K value in our experiment later.

Authorized licensed use limited to: INSTITUTE OF ENGINEERING & MANAGEMENT TRUST. Downloaded on April 24,2026 at 14:46:40 UTC from IEEE Xplore. Restrictions apply.

C. Enhancing Accuracy

Since outlier scores from KSE-test, the probability of instances being outliers, can mislabel normal instances as intrusions, we need an incremental step to improve the quality of the labeled result. By eliminating normal instances that are contaminated in outlier labeled data, we can reduce the number of mislabeled instances by using normal profile from previous step. We apply the normal profile to identify normal instances that are hiding in the outlier results.

D. Labeling clusters

In this step, we apply outlier detection result to labeled clusters by calculating the anomaly score for each cluster using simple majority vote approach. We assume that instances in the same cluster have high similarity to each other. To identify intrusion clusters, we applied the outlier detection result by counting the number of outlier instances in cluster using instance type form outlier detection results. If more than half of instances in the cluster are labeled as intrusions, this cluster is labeled as intrusion cluster. This idea is to reduce the effect of false labeled instances from outlier detection process and enhance an accuracy of our final results based on assumption that the size of clusters are small enough to provide high similarity in cluster.

IV. Evaluation

Our implementation was done in JAVA using the clustering algorithm from WEKA library [18]. All experiments were performed on Windows 7 64-bit version running on core i7-3610QM, 2.3GHz processor and 8GB memory DDR3 RAM.

A. Experiment Data

We use TCPdump data from KDD'99 (taken from KDD Cup 1999: computer network intrusion detection competition). This dataset has 41 attributes and approximately 5 million entries. It contains 24 attack types from 4 main categories. These attack types are: DoS, R2L, U2R, and probing. We remove flooding attacks from the dataset because they do not conform to our main assumption of anomaly detection that intrusion instances are few and different form normal instances. We resample 400,000 random records into the dataset that has 97 percent of normal instances and 3 percent of intrusion instances to satisfy our assumption.

B. Data Preprocessing

To prepare the dataset for outlier detection algorithm, we change all categorical data values into numerical values. After that, we remove the duplicate records and normalize all attribute values to be between 0 and 1. This step is to ease the calculation and measurement of distance between instances.

C. Measurement Metric

A confusion matrix, a calculated detection rate and a false positive rate, is used for evaluation. Confusion Matrix [19], also called a matching matrix, is used to show the efficiency of prediction result of algorithms by categorized the number of predicted instances using the actual class of those instances. Table, in figure 3, shows the confusion matrix. The rows represent the predicted labels, while the columns represent the actual class. The numbers of predicted instances are shown in each cell. These values are useful for describing the quality of algorithm. They provide the number of the correctly labeled and mislabeled instances. So, it can be shorten into a 2x2 table.

		Condition (as determined by "Gold standard")		
		Condition Positive	Condition Negative	
Test Outcome	Test Outcome Positive	True Positive	False Positive (Type I error)	Positive predictive value = Σ True Positive / Σ Test Outcome Positive
	Test Outcome Negative	False Negative (Type II error)	True Negative	Negative predictive value = Σ True Negative / Σ Test Outcome Negative
		Sensitivity = Σ True Positive / Σ Condition Positive	Specificity = Σ True Negative / Σ Condition Negative	

Figure 3:Table of confusion and relationships among terms [19]

The table of confusion contains the numbers that represent true positive (TP), true negative (TN), false positive (FP) and false negative (FN). True positive and true negative values are the numbers of instances that correctly identified as its actual type, attack instance marked as an intrusion and normal instance marked as a norm, respectively. On the other side, false positive and false negative values are the numbers of instances that identified to wrong types, attacks marked as the norm and normal instances marked as the intrusion. So, these values can be used to calculate the measure values for intrusion detection, as the following description.

True positive rate (TPR or sensitivity) is calculated from the number of the attack instances that are correctly identified divided by the number of all actual attacks in dataset.

$$TPR = \frac{TP}{TP + FN} \tag{3}$$

- True negative rate (TNR or specificity) is calculated from the number of the normal instances that correctly identified divided by the number of all actual normal instances in dataset.

$$TNR = \frac{TN}{TN + FP} \tag{4}$$

- False positive rate (FPR or false alarm rate) is calculated from the number of the normal instances marked as attacks divided by the number of all actual normal instances in dataset.

$$FPR = \frac{FP}{TN + FP} \tag{5}$$

- False negative rate (FNR) is calculated from the number of the attacks that are marked as normal divided by the number of all actual attacks in dataset.

$$FNR = \frac{FN}{TP + FN} \tag{6}$$

Authorized licensed use limited to: INSTITUTE OF ENGINEERING & MANAGEMENT TRUST. Downloaded on April 24,2026 at 14:46:40 UTC from IEEE Xplore. Restrictions apply.

- Positive predictive value (PPV or detection rate) is the number of the intrusive instances that are correctly identified as intrusion divided by the number of the instances predicted as intrusion.

$$PPV = \frac{TP}{TP + FP} \tag{7}$$

- Accuracy is the number of correctly identified instances divided by all instances.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{8}$$

All measure variables above can be used to represent the efficiency of our intrusion detection result. In our experiment we choose detection rate (DR), false positive rate (FPR) and accuracy (ACC) to excess performance of experiment result.

D. Experimental Result

In this section, we describe the result of our approach for KSE-test comparing to our proposed method. The first part of our experiment, we preprocessed and categorized the KDD'99 dataset using the traditional KSE-test by varying sample size among100, 300, 500, 700 and 1000 values. (In the experiment, sample1 size and sample2 size are the same value as defined by sample size). We vary the threshold value between 0.70 and 0.95 by incrementing the value by 0.05 per step in order to observe the effect of varying the threshold value. The result is shown in table 1.

The result shows that the quality of the result directly depends on the value of threshold used. We can see that high threshold values yield high detection rate and low false positive rate. However, using high threshold can drop the accuracy when only few numbers of instances were labeled. The sample size does not significantly impact the detection results.

In the second step, which shows the results of our proposed method, we use the result from the first step (the single KSE-test, sample size = 1000) to reduce mislabeling using normal profile from K-means clustering. This result can also be used to observe the different between the single traditional KSE-test and our proposed method.

By varying the K value (number of clusters produced from K-means algorithm), we can find the proper K that effect on the different number of clusters. We then calculate the detection rate, the false alarm rate and accuracy from the results obtained by varying the threshold and K value. The calculated results are in Table2.

Table 2 suggests that removing normal instance can yield better result (higher detection rate, lower false positive rate and higher accuracy). From our observation, the number of clusters (K) between 20 to 75 is the most suitable range for this dataset. As shown in the second to the last row in Table 2, the K value in this range can effectively identify intrusions (higher detection rate and lower false alarm rate) compared to the tradition KSE-test method (in the first experiment), which is the results shown in the first row in Table 2, and to those of other K-values with the same threshold.

From the experiment, we proposed the guideline for selecting the K-value as follows. For good clustering result, the largest cluster should not be larger than 20% and should not be smaller than 10% of dataset. It is worth clarifying that the proper K value might not necessary be equal to the real number of classes in dataset.

We can see that our proposed method, which contains addition step, can improve the quality of the result in terms of DR, FPR and ACC. The results in Table 2 clearly show that the values of threshold between 0.8 and 0.9 yield the best results. From the result, we can see that the decrease in the detection rate and the accuracy were less than when using the single KSE-test result at the first row. So we can select the lower threshold value without affecting the detection rate too much. This shows that our method supports wider range of thresholds. We can see that the range of values between 0.8 and 0.9 yields the best result. Thus, we suggest that the range (from 0.8 to 0.9) is the suitable range for this dataset. Using threshold in the range 0.8 to 0.9 and K value in the range 20 to 75, we found that the average results yield 95.5 percent detection rate with less than 1 percent of false with the 98 percent accuracy.

E. Complexity Analysis

In this section, we describe the complexity of our proposed method. Time complexity of KSE-test for a given sample size is a linear time of $O(C_1C_2n)$. In this case, n is the number of records in dataset S. C_1 and C_2 are sample size of S_1 and sample size of set S_2 respectively. (In our experiment, both samples size are equal). Time Complexity for K-means clustering is O(nKL) where n is number of data input, K is number of initial clusters and L is maximum number of iterations. Since the cost of remove normal instances and majority vote are O(n), the overall complexity of proposed method is O(n).

V. Conclusion

We applied K-means clustering and KS-test Outlier detection to the TCPdump log dataset (KDD'99). Our proposed

TABLE I. SINGLE KSE-TEST VARYING SAMPLE SIZE

Threshold	Sample Size														
	100			300			500			700			1000		
	DR	FPR	ACC	DR	FPR	ACC	DR	FPR	ACC	DR	FPR	ACC	DR	FPR	ACC
0.70	0.256318	0.077209	0.920920	0.247189	0.082479	0.916265	0.207921	0.105714	0.894375	0.225206	0.094353	0.905080	0.212701	0.102438	0.897480
0.75	0.305735	0.049923	0.942900	0.283268	0.063647	0.932663	0.257669	0.076085	0.921815	0.264797	0.073183	0.924580	0.259069	0.076070	0.922013
0.80	0.262049	0.041080	0.944303	0.252499	0.044910	0.941153	0.305731	0.054454	0.940440	0.308344	0.052361	0.941853	0.304649	0.053732	0.940715
0.85	0.726610	0.004541	0.977303	0.891253	0.001541	0.980758	0.819550	0.002997	0.980298	0.819550	0.002997	0.980298	0.278650	0.037183	0.947865
0.90	0.994650	0.000036	0.976473	0.995289	0.000028	0.975783	0.995325	0.000028	0.975828	0.995325	0.000028	0.975828	0.993542	0.000072	0.980700
0.95	0.995640	0.000008	0.971705	0.967742	0.000008	0.970218	0.990762	0.000010	0.971063	0.990762	0.000010	0.971063	0.996896	0.000013	0.974003

Authorized licensed use limited to: INSTITUTE OF ENGINEERING & MANAGEMENT TRUST. Downloaded on April 24,2026 at 14:46:40 UTC from IEEE Xplore. Restrictions apply.

TABLE II. COMPARING THE TRADITIONAL KSE-TEST TO OUR PROPOSED METHOD (VARYING THE K VALUE)

Experiment	Threshold																	
	0.7			0.75			0.8			0.85			0.9			0.95		
	DR	FPR	ACC	DR	FPR	ACC	DR	FPR	ACC	DR	FPR	ACC	DR	FPR	ACC	DR	FPR	ACC
KSE-test	0.212701	0.102438	0.897480	0.259069	0.076070	0.922013	0.304649	0.053732	0.940715	0.278650	0.037183	0.947865	0.993542	0.000072	0.980700	0.996896	0.000013	0.974003
K=5	0.228931	0.091165	0.907825	0.266599	0.072454	0.925267	0.307752	0.052843	0.941530	0.279729	0.036951	0.948078	0.994001	0.000067	0.980705	0.997516	0.000010	0.974005
K=10	0.191919	0.080825	0.910220	0.217068	0.068121	0.922243	0.238950	0.051789	0.935538	0.279871	0.036799	0.948177	0.994230	0.000064	0.980708	0.998135	0.000008	0.974008
K=20	0.634643	0.011889	0.978500	0.815366	0.004639	0.985373	0.948749	0.001098	0.988650	0.979927	0.000268	0.982433	0.997728	0.000023	0.979857	0.997863	0.000008	0.973495
K=30	0.664096	0.010441	0.979895	0.811849	0.004747	0.985265	0.948077	0.001113	0.988640	0.978240	0.000291	0.982417	0.997726	0.000023	0.979847	0.997857	0.000008	0.973485
K=40	0.641916	0.008054	0.976192	0.868121	0.002149	0.981640	0.934469	0.000982	0.982630	0.970315	0.000428	0.983150	0.997674	0.000026	0.980698	0.998135	0.000008	0.974008
K=50	0.442933	0.018366	0.966350	0.672977	0.006946	0.977128	0.853051	0.002433	0.981340	0.956583	0.000637	0.982988	0.997214	0.000031	0.980708	0.997516	0.000010	0.974005
K=75	0.492803	0.022070	0.969393	0.664856	0.010652	0.980165	0.829971	0.004258	0.986030	0.949896	0.000742	0.982930	0.996982	0.000034	0.980703	0.996896	0.000013	0.974003
K=100	0.357119	0.015093	0.963493	0.448769	0.009928	0.968210	0.498754	0.007776	0.969963	0.583066	0.005369	0.972075	0.991727	0.000044	0.975053	0.995327	0.000010	0.972120
K=125	0.336839	0.015928	0.962398	0.420518	0.010729	0.967145	0.461286	0.008894	0.968760	0.647897	0.004077	0.973323	0.991241	0.000046	0.975048	0.994166	0.000013	0.972118
K=150	0.354315	0.017317	0.962420	0.406648	0.013802	0.965788	0.452554	0.009918	0.968332	0.647444	0.004160	0.973375	0.991296	0.000046	0.975080	0.994166	0.000013	0.972118
K=175	0.364882	0.022224	0.960828	0.464933	0.014314	0.968180	0.498056	0.010649	0.969920	0.624081	0.004611	0.972953	0.986532	0.000072	0.975057	0.994166	0.000013	0.972118
K=200	0.247951	0.029330	0.950930	0.321479	0.019719	0.959935	0.382470	0.012039	0.965555	0.436838	0.005067	0.968898	0.983710	0.000046	0.972673	0.994166	0.000013	0.972118
K=500	0.288295	0.021830	0.957402	0.358035	0.015763	0.963238	0.451604	0.010134	0.968265	0.614747	0.004309	0.972490	0.985823	0.000072	0.974798	0.977778	0.000013	0.970538
K=800	0.229787	0.020206	0.956248	0.290416	0.014503	0.961690	0.356545	0.009121	0.966055	0.534542	0.002448	0.970353	0.967330	0.000059	0.971645	0.977578	0.000013	0.970533
K=1000	0.228319	0.021856	0.955073	0.268807	0.017260	0.959413	0.324365	0.011585	0.964158	0.471553	0.004237	0.969558	0.972289	0.000059	0.971960	0.977578	0.000013	0.970533

a. The result of the traditional KSE-test (without additional method) is shown in the first row as a baseline of the experiment

b. The other rows are the result of our proposed method (Varying the K Value).

method can compute in linear time while still maintaining high detection rate and low false positive rate (about 95 percent detection rate and less than 1 percent false positive rate with the accuracy of 98 percent) when averaging from results in the suggested range of threshold and K value. The time it takes to compute the result does not grow exponentially to the size of input data. The proposed method can solve the performance and scalability problem in order to support a scalable log analysis or large scale intrusion detection.

In the future, we aim at improving the performance of our approach by making it faster. In particular, we will optimize and parallel our method. Our goal is to apply this unsupervised intrusion detection approach to practically use in large scale log analysis for effective intrusion detection system.

ACKNOWLEDGMENT

The Scholarship from the Graduate School, Chulalongkorn University to commemorate the 72nd anniversary of his Majesty King Bhumibala Aduladeja is gratefully acknowledged.

REFERENCES

[1] F. Sabahi and A. Movaghar, "Intrusion Detection: A Survey," in Systems and Networks Communications, 2008. ICSNC '08. 3rd International Conference on, 2008, pp. 23-26.

[2] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," ACM Comput. Surv., vol. 41, pp. 1-58, 2009.

[3] D. K. Denatious and A. John, "Survey on data mining techniques to enhance intrusion detection," in Computer Communication and Informatics (ICCCI), 2012 International Conference on, 2012, pp. 1-5.

[4] M. Ahmed and A. Naser, "A novel approach for outlier detection and clustering improvement," in Industrial Electronics and Applications (ICIEA), 2013 8th IEEE Conference on, 2013, pp. 577-582.

[5] M. Goldstein and A. Dengel, "Histogram-based outlier score (hbos): A fast unsupervised anomaly detection algorithm."

[6] E. M. Knorr and R. T. Ng, "A unified approach for mining outliers," presented at the Proceedings of the 1997 conference of the Centre for Advanced Studies on Collaborative research, Toronto, Ontario, Canada, 1997.

[7] E. M. Knorr and R. T. Ng, "Algorithms for Mining Distance-Based Outliers in Large Datasets," presented at the Proceedings of the 24rd International Conference on Very Large Data Bases, 1998.

[8] E. M. Knorr, R. T. Ng, and V. Tucakov, "Distance-based outliers: algorithms and applications," The VLDB Journal, vol. 8, pp. 237-253, 2000.

[9] S. Ramaswamy, R. Rastogi, and K. Shim, "Efficient algorithms for mining outliers from large data sets," presented at the Proceedings of the 2000 ACM SIGMOD international conference on Management of data, Dallas, Texas, USA, 2000.

[10] S. D. Bay and M. Schwabacher, "Mining distance-based outliers in near linear time with randomization and a simple pruning rule," presented at the Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining, Washington, D.C., 2003.

[11] H. Fan, O. Zaïane, A. Foss, and J. Wu, "A Nonparametric Outlier Detection for Effectively Discovering Top-N Outliers from Engineering Data," in Advances in Knowledge Discovery and Data Mining. vol. 3918, W.-K. Ng, M. Kitsuregawa, J. Li, and K. Chang, Eds., ed: Springer Berlin Heidelberg, 2006, pp. 557-566.

[12] M. M. Breunig, H.-P. Kriegel, R. T. Ng, #246, and r. Sander, "LOF: identifying density-based local outliers," presented at the Proceedings of the 2000 ACM SIGMOD international conference on Management of data, Dallas, Texas, USA, 2000.

[13] M. Ankerst, M. M. Breunig, H.-P. Kriegel, #246, and r. Sander, "OPTICS: ordering points to identify the clustering structure," SIGMOD Rec., vol. 28, pp. 49-60, 1999.

[14] J. Tang, Z. Chen, A.-c. Fu, and D. Cheung, "Enhancing Effectiveness of Outlier Detections for Low Density Patterns," in Advances in Knowledge Discovery and Data Mining. vol. 2336, M.-S. Chen, P. Yu, and B. Liu, Eds., ed: Springer Berlin Heidelberg, 2002, pp. 535-548.

[15] M. A. Dalal and N. D. Harale, "A survey on clustering in data mining," presented at the Proceedings of the International Conference & Workshop on Emerging Trends in Technology, Mumbai, Maharashtra, India, 2011.

[16] L. Zhengjie, L. Yongzhong, and X. Lei, "Anomaly Intrusion Detection Method Based on K-Means Clustering Algorithm with Particle Swarm Optimization," in Information Technology, Computer Engineering and Management Sciences (ICM), 2011 International Conference on, 2011, pp. 157-161.

[17] M. S. Kim, "Robust, Scalable Anomaly Detection for Large Collections of Images," in Social Computing (SocialCom), 2013 International Conference on, 2013, pp. 1054-1058.

[18] (2015). WEKA. Available: http://www.cs.waikato.ac.nz/ml/weka/

[19] (2015). KDD Cup 1999 Computer network intrusion detection. Available: http://sigkdd.org/kdd-cup-1999-computer-network-intrusion-detection

[20] (2015). Confusion matrix. Available: http://en.wikipedia.org/wiki/Confusion_matrix

Authorized licensed use limited to: INSTITUTE OF ENGINEERING & MANAGEMENT TRUST. Downloaded on April 24,2026 at 14:46:40 UTC from IEEE Xplore. Restrictions apply.